μMPC: Safe Nonlinear and World Model Control in Microseconds via Koopman-Linearized Precompilation

Devesh Nath Glen Chou

***Abstract*— We present μMPC, a fast Model Predictive Control (MPC) framework based on learned Koopman-linearized dynamics. By precompiling finite-horizon maps from control sequences to predicted physical outputs, μMPC analytically eliminates lifted-state trajectories from online optimization, yielding an action-only Koopman-Compiled QP (KC-QP) with Hn_u decision variables that is independent of lifted dimension. To efficiently solve the KC-QP for fixed horizon, quadratic weights, and primal regularization, we precompute the primal operator offline, reducing online primal-dual iterations to matrix-vector products and projections while accommodating changing goals and constraints without refactorization. This efficiency further enables scalable Koopman dynamics learning through a differentiable KC-QP that directly supervises finite-horizon control recovery, paired with batched DAgger-style data aggregation. μMPC achieves 4096-dimensional whole-body quadruped control in under 5 ms, sub-millisecond planning with 2048-dimensional visual world models and improved task success, and $H = 75$ unconstrained MPC onboard a Crazyflie microcontroller in approximately 24 ms. GPU-batched iterative best response scales collision-aware planning to 200 quadrotors at over 30 Hz. Across benchmarks, μMPC reduces planning latency by one to over two orders of magnitude, demonstrating that high-dimensional learned dynamics need not come at the expense of fast, real-time optimization.**

Project Website Code Video

The authors are with the Georgia Institute of Technology, Atlanta, GA 30308, USA. {dnath7,chou}@gatech.edu

I. Introduction

Applying optimal control to complex robotic systems requires expressive dynamics models and fast solvers for real-time deployment. Meeting both demands is challenging, particularly for resource-constrained or high-dimensional systems. Embedded aerial robots face strict compute, memory, and power budgets [1], [2]; whole-body legged robot control requires real-time optimization over high-dimensional dynamics [3], [4]; visuomotor planning relies on learned world models with thousands of latent states [5], [6]; and multi-agent systems require many concurrent optimizations [7], [8]. In these settings, controllers must plan over long horizons, react rapidly to observations, and satisfy state, task, safety, and actuation constraints. Nonlinear Model Predictive Control (MPC) provides a general framework for achieving this, but its online computational cost remains scales with model dimension, horizon length, and constraint complexity despite recent advances in sparsity exploitation, factorization caching, and GPU parallelism [3], [4].

This computational challenge is particularly pronounced for learned dynamics and world models. While these models can capture complex dynamics from high-dimensional observations, planning often requires repeated computationally-expensive evaluations of candidate action sequences through the learned dynamics [5], [9], [10]. One approach to reducing this cost is to learn dynamics that are linear in a lifted space,

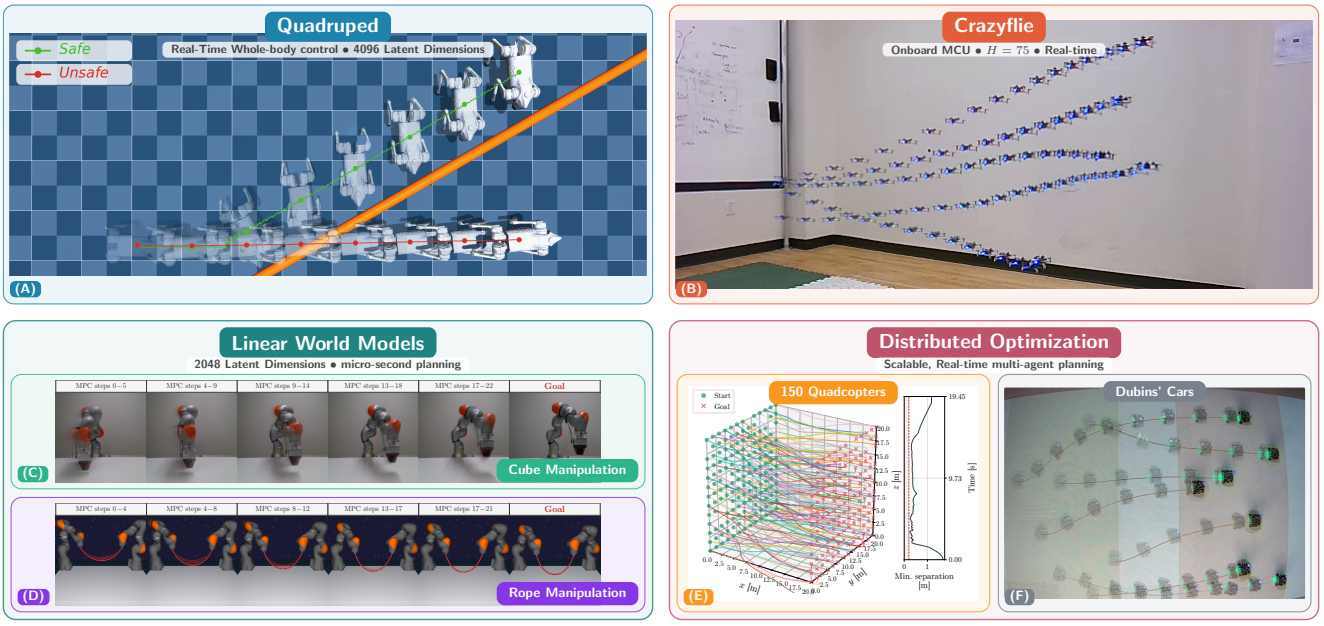

Fig. 1. **μMPC** enables real-time control from microcontrollers to massive learned models and multi-agent systems: **(A)** quadruped safety filtering with a 4096-dimensional ($\sim$16M-parameter) Koopman model; **(B)** onboard $H = 75$ Crazyflie MPC; **(C–D)** hardware cube and rope manipulation; **(E)** collision-constrained planning for 150 quadrotors; and **(F)** seven-robot multi-agent planning on the Robotarium.

which enables finite-horizon predictions to be expressed directly as a function of the control inputs [11]. However, this alone does not ensure efficient optimization, since MPC often requires iterative relinearization of nonlinear constraints, modifying the optimization and necessitating expensive matrix refactorizations. This motivates an MPC framework that moves expensive computations offline while allowing goals and constraints to be updated efficiently online.

Another challenge is learning dynamics for control rather than prediction alone. Conventional Koopman training emphasizes state-prediction accuracy [12], but low rollout error does not guarantee that optimizing the learned dynamics recovers the correct control sequence. Differentiable optimization exposes this sensitivity [13], but repeatedly solving and differentiating through optimization problems can be prohibitively expensive at scale. Moreover, models trained on fixed demonstrations fail to account for controller-induced distribution shift, where prediction errors can compound. Effective learned MPC therefore requires direct control-level supervision and adaptation to the controller's own state distribution. Our low-impact, batched differentiable QPs enable scalable control-recovery supervision alongside state prediction, coupled with fast DAgger-style aggregation during training.

To address these challenges, we present μMPC, a learned MPC framework for nonlinear systems that exploits approximate Koopman-linearized dynamics to formulate Koopman-Compiled Quadratic Programs (KC-QPs) for MPC which can be solved in microseconds. To solve these KC-QPs, μMPC employs a tailored primal-dual solver with a fixed primal operator precomputed offline, reducing constrained online optimization to a computationally efficient loop of matrix-vector products and projections. The compiled primal operator is reusable across changing constraints, goals and MPC steps, drastically reducing online computation. In addition to standard state reconstruction and latent consistency supervision for Koopman representation learning, we differentiate through a KC-QP built for scalable control-recovery supervision, directly training the model for its finite-horizon control response. Paired with fast and parallelized KC-QP solves during training, DAgger-style aggregation enables scalable on-policy data generation for coverage over the controller-induced distribution. As a result, μMPC bridges scalable learning and real-time optimal control, enabling large-scale Koopman representations to be trained for their control response while keeping online optimization computationally efficient. Our contributions are:

- **μMPC** compiles learned Koopman dynamics into an action-only Koopman-Compiled Quadratic Program (KC-QP) with a fixed, precomputed primal operator. This enables fast primal-dual iterations for constrained optimization requiring only matrix-vector products and projections, while accommodating changing goals and constraints without online refactorization.
- We leverage the compact, parallelizable structure of KC-QPs to define a differentiable control-recovery objective that directly trains learned dynamics to recover demonstrated control sequences. Batched KC-QP solves further enable scalable DAgger-style data aggregation to mitigate controller-induced distribution shift.
- We demonstrate μMPC across embedded, high-dimensional, distributed, and visual control, achieving microsecond-scale MPC on classical systems, unconstrained $H = 75$ MPC onboard a Crazyflie MCU in 24 ms, 4096-dimensional whole-body quadruped control in under 5 ms, and real-time collision-aware planning for 200 quadrotors with hardware validation on 7 hardware differential-drive robots. We further train 2048-dimensional linear world models for sub-millisecond visual planning, outperforming evaluated visual-planning baselines on Reacher and bimanual rope manipulation, alongside a hardware cube-manipulation demonstration.

II. Related Work

Koopman MPC and fast optimal control. Koopman methods lift nonlinear dynamics into approximately linear coordinates for linear MPC [11], [14], with recent extensions to robotic systems [15], [16]. Existing formulations retain important online dependencies: Korda and Mezić [11] use a prescribed lifting basis, so nonlinear costs and constraints must be expressible through the chosen lifted/output coordinates. Wu et al. [17] condense Koopman MPC but specialize to input constraints; and Koopman-BoxQP [18] eliminates high-dimensional observables but keeps physical states and controls as decision variables under box structure. More broadly, fast MPC exploits sparsity, condensing, cached factorizations, and specialized solvers [1], [19], [20], while recent GPU methods parallelize dynamics and optimization [2]–[4]. In contrast, μMPC compiles Koopman dynamics into an action-only KC-QP. For fixed horizon, quadratic weights, and primal regularization, its precomputed primal operator remains reusable across changing goals and constraints, reducing online constrained optimization to matrix–vector products and projections without refactorization.

Differentiable control learning. Differentiable optimization and MPC support end-to-end learning through constrained control [13], [21]–[23]. KCPO [24] combines deep Koopman dynamics with differentiable MPC under hard box control constraints. We instead differentiate an efficient control-recovery KC-QP that directly supervises the finite-horizon control response, then pair it with DAgger-style on-policy aggregation. This targets control relevant model error and controller-induced distribution shift without differentiating through a full nonlinear MPC solve.

World models and latent planning. World models plan through learned latent dynamics [5], [9], [25]–[27]. DINO-WM and LeWorldModel optimize action sequences through repeated latent prediction; Fast-LeWM parallelizes prediction but retains CEM search [10]; learned planners instead amortize refinement from imagined rollouts [28]. μMPC instead compiles the full-horizon response into a compact KC-QP for sub-millisecond planning, replacing repeated latent rollout/search at planning time while permitting explicit linearized state/task constraints and action bounds.

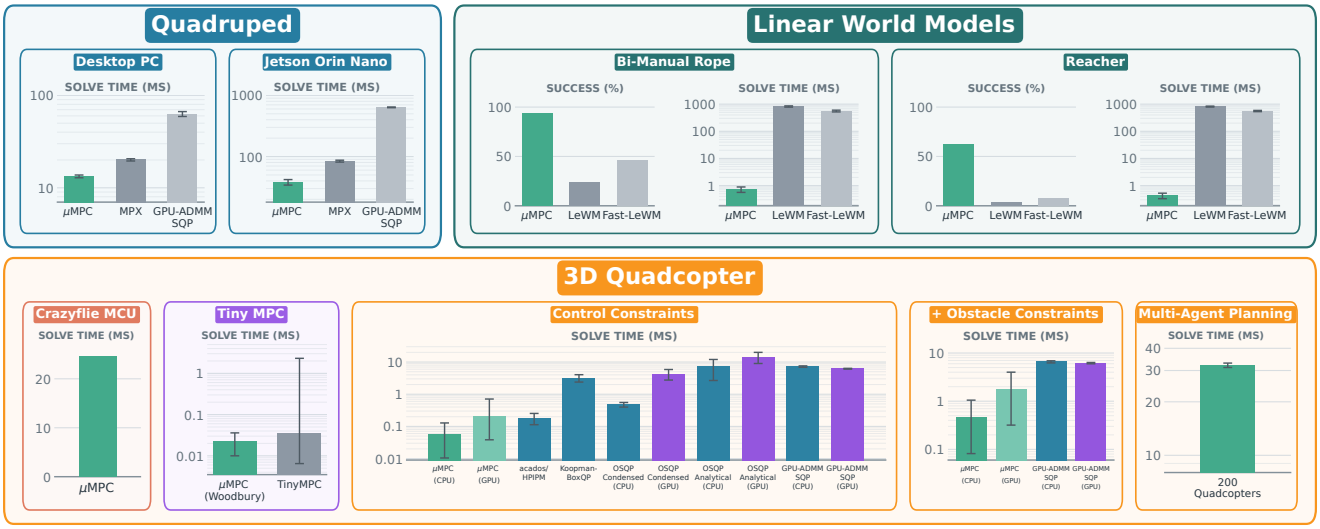

Fig. 2. μMPC performance across model, hardware, and problem scale: low-latency quadruped and visual-world-model control, onboard microcontroller MPC, constrained 3D quadrotor control, and real-time multi-agent planning. μMPC (Woodbury) reports unconstrained Real-Time Iteration (RTI) times for comparisons against Tiny-MPC. Solve-time axes are logarithmic where indicated to accommodate the wide latency range across methods.

III. Problem Statement

We consider a discrete-time nonlinear system

$$x_{k+1} = f(x_k, u_k), \qquad x_k \in \mathbb{R}^{n_x},\ u_k \in \mathbb{R}^{n_u}. \tag{1}$$

At each control step, we seek a horizon-H input sequence $U = [u_0^\top \ \cdots \ u_{H-1}^\top]^\top$ that drives a task-relevant output y_k toward a goal y^{goal} while satisfying state, task, and input constraints. Locally linearizing the nonlinear constraints about the current nominal trajectory gives

$$\begin{aligned} \underset{x_{1:H},\,u_{0:H-1}}{\text{minimize}} \quad & \sum_{k=1}^{H-1} \|y_k - y^{\text{goal}}\|^2_{Q_{y,k}} + \sum_{k=0}^{H-1} \|u_k\|^2_{R_k} + \|y_H - y^{\text{goal}}\|^2_{Q_f} \\ \text{subject to} \quad & x_{k+1} = f(x_k, u_k), \quad k = 0, \dots, H-1, \\ & D_{y,k} y_{k+1} + D_{u,k} u_k \le b_k, \quad k = 0, \dots, H-1, \\ & D_f y_H \le b_f. \end{aligned} \tag{2}$$

Here $y_k = S_y x_k$ selects task-relevant coordinates, with $S_y = I$ recovering the full state. Linearization offsets are absorbed into b_k and b_f, while higher-order error is handled through the tightenings in Sec. IV-C. Since (2) remains nonlinear through f, we seek a approximately linear lifted predictive model that moves expensive optimization algebra offline, leaving only fixed linear algebra and projections online.

IV. Method

Our method proceeds in four stages. First, we learn a nonlinear lifting with control affine linear latent dynamics. Second, we compile its finite-horizon response into precomputed maps and eliminate the latent trajectory from the optimization variables. Third, we solve the resulting action-only KC-QP with a primal–dual method whose primal update operator is fixed for a given model, horizon, objective, and primal regularization, and can therefore be precomputed offline and reused as constraints are relinearized online. Finally, we train the representation with differentiable QP for direct control supervision, and DAgger aggregation to improve its finite-horizon control behavior on the distribution induced by its own controller.

A. Learned Dynamics

In Koopman models [12], the physical state x_k is encoded as

$$z_k = \text{enc}_\theta(x_k), \qquad z_k \in \mathbb{R}^{n_z}, \tag{3}$$

with latent state z_k, learned globally linear latent dynamics

$$z_{k+1} = Az_k + Bu_k, \qquad A \in \mathbb{R}^{n_z \times n_z},\ B \in \mathbb{R}^{n_z \times n_u}, \tag{4}$$

and affine decoder

$$x_k = Cz_k + c, \qquad C \in \mathbb{R}^{n_x \times n_z},\ c \in \mathbb{R}^{n_x}. \tag{5}$$

At each MPC step only the measured physical state is encoded, $z_0 = \text{enc}_\theta(x_0)$; all subsequent predictions follow from the linear latent rollout and decoding into the physical state space in (4). For notational brevity, we omit prediction hats and use x_k (and similarly y_k) for these decoded predictions when the distinction is clear from context. When measured or demonstrated future states appear alongside predictions, we distinguish them explicitly.

B. Precomputed Maps

To accelerate online computation, all horizon operators that depend only on A, B, C, and H are constructed offline.

1) Condensed latent and physical prediction: Define

$$U = \left[u_0^\top \cdots u_{H-1}^\top\right]^\top \in \mathbb{R}^{Hn_u}, \quad Z = \left[z_1^\top \cdots z_H^\top\right]^\top \in \mathbb{R}^{Hn_z}. \tag{6}$$

Repeated application of (4) yields

$$Z = \Phi z_0 + \Gamma U, \tag{7}$$

where

$$\Phi = \begin{bmatrix} A \\ A^2 \\ \vdots \\ A^H \end{bmatrix}, \qquad \Gamma = \begin{bmatrix} B & 0 & \cdots & 0 \\ AB & B & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ A^{H-1}B & A^{H-2}B & \cdots & B \end{bmatrix}. \tag{8}$$

Thus the latent trajectory is affine in only (z_0, U). Stacking $X = [x_1^\top \ \cdots \ x_H^\top]^\top$ and applying (5) yields

$$X = a_x(z_0) + G_x U \tag{9}$$

with,

$$a_x(z_0) = (I_H \otimes C)\Phi z_0 + \mathbf{1}_H \otimes c, \quad G_x = (I_H \otimes C)\Gamma. \tag{10}$$

Hence neither Z nor X must appear as optimization variables since they are defined implicitly through U. Equations (7)–(8) are used only for exposition. In implementation, we never materialize the full Φ or Γ matrices. We construct the projected operators directly from the block products.

2) Finite Horizon output maps: Let $S_y \in \mathbb{R}^{n_y \times n_x}$ select task-relevant coordinates explicitly represented in the decoded physical feature vector x_k used by a cost or constraint and define

$$C_y = S_y C, \qquad c_y = S_y c, \qquad y_k = C_y z_k + c_y. \tag{11}$$

For $Y = [y_1^\top \ \cdots \ y_H^\top]^\top$

$$Y = a_y(z_0) + G_y U \tag{12}$$

with,

$$\begin{aligned} &a_y(z_0) = F_y z_0 + \mathbf{1}_H \otimes c_y, \quad F_y = (I_H \otimes C_y)\Phi, \\ &G_y = (I_H \otimes C_y)\Gamma. \end{aligned} \tag{13}$$

The operators F_y and G_y are fixed after training and are precomputed for each required output selector.

C. Koopman Compiled Quadratic Program (KC-QP)

1) Linear surrogate OCP: We replace the nonlinear rollout in (2) by the learned linear dynamics while retaining the locally linearized constraints:

$$\begin{aligned} \underset{z_{1:H},u_{0:H-1}}{\text{minimize}} \quad & \sum_{k=1}^{H-1} \|y_k - y^{\text{goal}}\|^2_{Q_{y,k}} + \sum_{k=0}^{H-1} \|u_k\|^2_{R_k} + \|y_H - y^{\text{goal}}\|^2_{Q_f} \\ \text{subject to} \quad & z_{k+1} = Az_k + Bu_k, \quad k = 0,\dots,H-1, \\ & D_{y,k} y_{k+1} + D_{u,k} u_k + \Delta_k \le b_k, \quad k = 0,\dots,H-1, \\ & D_f y_H + \Delta_f \le b_f. \end{aligned} \tag{14}$$

The nonnegative margins Δ_k, Δ_f tighten linearized constraints for approximation error, combining linearization and model mismatch terms. Since (14) is a QP, substituting the precomputed maps eliminates $z_{1:H}$ from the decision variables.

2) Condensed objective and constraints: Let $q = Hn_u$, $Y^{\text{goal}} = \mathbf{1}_H \otimes y^{\text{goal}}$, $\bar{Q}_y = \text{blkdiag}(Q_{y,1},\dots,Q_{y,H-1},Q_f)$, and $\bar{R} = \text{blkdiag}(R_0,\dots,R_{H-1})$. The quadratic objective becomes

$$\begin{aligned} J(U) &= \tfrac{1}{2}\|a_y(z_0) + G_y U - Y^{\text{goal}}\|^2_{\bar{Q}_y} + \tfrac{1}{2}\|U\|^2_{\bar{R}} \\ &= \tfrac{1}{2} U^\top \mathscr{H} U + g^\top U + c, \end{aligned} \tag{15}$$

with

$$\begin{aligned} &\mathscr{H} = G_y^\top \bar{Q}_y G_y + \bar{R}, \quad g = G_y^\top \bar{Q}_y \big(a_y(z_0) - Y^{\text{goal}}\big), \\ &c = \frac{1}{2}\big(a_y(z_0) - Y^{\text{goal}}\big)^\top \bar{Q}_y \big(a_y(z_0) - Y^{\text{goal}}\big). \end{aligned} \tag{16}$$

Since $c(z_0)$ is independent of U, it is dropped. For a fixed model, horizon, and cost weights, changing the goal modifies only the linear term g (and the discarded constant term), while the Hessian $\mathscr{H}$ and its factorization remain unchanged. Stacking stage, terminal, and input inequalities gives $\bar{D}_y Y + \bar{D}_u U \le \bar{b} - \bar{\Delta}$. Using (12),

$$(\bar{D}_y G_y + \bar{D}_u) U \le \bar{b} - \bar{\Delta} - \bar{D}_y a_y(z_0), \qquad MU \le r(z_0), \tag{17}$$

where $M = \bar{D}_y G_y + \bar{D}_u$ and $r(z_0) = \bar{b} - \bar{\Delta} - \bar{D}_y a_y(z_0)$. The complete online QP is therefore

$$\begin{aligned} \underset{U \in \mathbb{R}^q}{\text{minimize}} \quad & \frac{1}{2} U^\top \mathscr{H} U + g(z_0)^\top U \\ \text{subject to} \quad & MU \le r(z_0). \end{aligned} \tag{18}$$

Crucially, only U is optimized; the latent and physical trajectories have been analytically eliminated.

3) Fixed Operator primal–dual solve: We efficiently solve (18) with a tailored primal-dual solver. We introduce $\lambda \ge 0$ for (18). With dual step $\sigma > 0$, primal regularization $\rho > 0$, and iterate U^j, the projected dual step is

$$\lambda^{j+1} = \big[\lambda^j + \sigma\big(MU^j - r(z_0)\big)\big]_+. \tag{19}$$

The corresponding proximal primal update solves

$$\begin{aligned} U^{j+1} = \underset{U}{\text{argmin}} \ & \tfrac{1}{2} U^\top \mathscr{H} U + g(z_0)^\top U + \lambda^{j+1\top} MU \\ & + \tfrac{\rho}{2}\|U - U^j\|_2^2, \end{aligned} \tag{20}$$

whose stationarity condition is

$$(\mathscr{H} + \rho I_q) U^{j+1} = \rho U^j - g(z_0) - M^\top \lambda^{j+1}. \tag{21}$$

Defining the constant primal block of the proximal KKT system

$$K = \mathscr{H} + \rho I_q, \tag{22}$$

gives the online update

$$U^{j+1} = K^{-1}\Big[\rho U^j - g(z_0) - M^\top \lambda^{j+1}\Big]. \tag{23}$$

Because K depends only on the learned model, horizon, objective weights, and ρ, the corresponding primal operator can be precomputed offline and reused across MPC iterations. Our implementations precompute and store the dense inverse K^{-1}, so (23) becomes a dense matrix–vector product. SQP may change the local constraint directions and therefore M, but this changes only MU^j, $M^\top \lambda^{j+1}$, and $r(z_0)$; no online inversion or factorization is required. We set $\sigma = 0.99\rho/\|M\|_F^2$, which guarantees $\sigma\|M\|_2^2/\rho < 1$.

Our stopping criteria use

$$\begin{aligned} r_{\text{feas}}^{j+1} &= \|\big(MU^{j+1} - r(z_0)\big)_+\|_2, \\ r_{\text{fp}}^{j+1} &= \big(\rho^2\|U^{j+1} - U^j\|_2^2 + \sigma^{-2}\|\lambda^{j+1} - \lambda^j\|_2^2\big)^{1/2}. \end{aligned} \tag{24}$$

For each linearized KC-QP, iterations stop when residuals meet tolerance or a real-time iteration budget is reached. If the budget is exhausted, the current iterate is returned and may remain suboptimal and may retain constraint violation.

D. Scalability

The KC-QP decision dimension is $q = Hn_u$, independent of the lifted dimension n_z, compared to $Hn_z + Hn_u$ variables in a non-condensed formulation. The fixed primal operator is precomputed offline in $\mathscr{O}(q^3)$ time with $\mathscr{O}(q^2)$ storage. Our implementations precompute the dense inverse K^{-1}, so each primal update is a dense matrix-vector product; on memory-constrained hardware, the fixed operator can instead be stored through a factorization. For m inequalities, each primal-dual iteration requires only $\mathscr{O}(q^2 + mq)$ work. With N_{PD} inner iterations and N_{SQP} constraint-linearization iterations, the online optimization complexity is

$$\mathscr{O}\big(N_{\text{SQP}} N_{\text{PD}} (q^2 + mq)\big), \tag{25}$$

excluding constraint linearization and model evaluation. These matrix-vector operations admit logarithmic-depth GPU reductions, although CPUs are often faster for small KC-QPs due to GPU launch, synchronization, and data transfer overhead, while larger batched problems benefit from parallelism.

The lifted dimension enters online through encoding and affine prediction terms which still scale with n_z. Thus, increasing representation capacity does not enlarge the KC-QP decision vector or its primal operator, although encoding and prediction-map evaluation retain a dependence on n_z.

E. Training via Differentiable QP and DAgger

We train on demonstrations (x_0, U, X), with $U = [u_0^\top \ \cdots \ u_{H-1}^\top]^\top$ and $X = [x_1^\top \ \cdots \ x_H^\top]^\top$. Rolling out the learned dynamics under U yields $\hat{z}_{1:H}$ and $\hat{x}_{1:H}$, while demonstrated states are encoded as $z_{1:H} = \mathrm{enc}_\theta(X)$. We initialize $A = I$ and B with Xavier initialization to avoid initially amplifying latent dynamics or control couplings. We use the multi-step losses

$$\mathscr{L}_x = \frac{1}{H}\sum_{k=1}^{H} \|\hat{x}_k - x_k\|_2^2, \quad \mathscr{L}_z = \frac{1}{H}\sum_{k=1}^{H} \|\hat{z}_k - z_k\|_2^2. \tag{26}$$

To supervise finite-horizon control response, we solve

$$\widehat{U} = \underset{V}{\operatorname{argmin}} \ \frac{1}{2}\|a_x(z_0) + G_x V - X\|_{\bar{Q}_{\text{track}}}^2 + \frac{1}{2}\|V\|_{\bar{R}_{\text{track}}}^2, \tag{27}$$

and optimize

$$\mathscr{L}_u = \|\widehat{U} - U\|_2^2, \qquad \mathscr{L} = \lambda_x \mathscr{L}_x + \lambda_z \mathscr{L}_z + \lambda_u \mathscr{L}_u. \tag{28}$$

Positive control regularization makes (27) strongly convex with a unique solution. Because it uses the controller's condensed map G_x, gradients through $\widehat{U}$ directly supervise the finite-horizon control sensitivity of the encoder and (A, B, C, c). Crucially, the objective is goal and constraint independent, training the dynamics to recover the demonstrated controls required to reproduce X.

To mitigate controller-induced distribution shift, we use DAgger-style aggregation. At each epoch, the finite-horizon tracking QP used for control-recovery supervision generates control sequences that are executed in simulation. The resulting trajectories are added to a bounded replay buffer and mixed with original demonstrations, adapting the model to controller-visited states while retaining expert data.

V. Experiments

Unless noted otherwise, models are trained on an NVIDIA RTX 4090 without state normalization using state, latent, and diffQP loss weights of 1, 0.1, and 10^{-2}. Desktop experiments use an i9-14900KF and RTX 4090 with JAX dense linear algebra. unless stated otherwise. Reported times are end-to-end MPC latency. We evaluate three claims: (i) KC-QP reduces constrained MPC latency across desktop, edge, and MCU hardware; (ii) DiffQP and DAgger improve finite-horizon control quality; and (iii) the formulation scales to high-dimensional whole-body and visual models and batched collision-constrained multi-agent planning. Constrained deployments terminate at residual tolerance or a fixed real-time iteration budget.

A. Dubins Car

We first consider a Dubins car with state $x = [p_x, p_y, \sin\theta, \cos\theta] \in \mathbb{R}^4$ and control $u = [v, \omega] \in \mathbb{R}^2$. We learn a 32-dimensional Koopman model using a three-hidden-layer, width-32 GELU encoder, trained for 250 epochs on 55-step rollouts with batch size 8192 using AdamW. Control-constrained Dubins MPC requires $0.054 \pm 0.018/0.066 \pm 0.019/0.090 \pm 0.028$ ms on CPU for $H = 15/30/45$, respectively, while GPU latency is $1.426 \pm 0.049/1.436 \pm 0.053/1.438 \pm 0.058$ ms, dominated by launch overhead for these small action-space QPs.

1) Real-Time Multi-Agent Planning: We deploy the learned model on seven differential-drive robots in a $3.2\,\mathrm{m} \times 2.0\,\mathrm{m}$ workspace (Fig. 1F). Each robot solves an $H = 28$ KC-QP with the other robots' predicted trajectories as time-varying collision constraints. We perform five IBR rounds per control update, with each KC-QP terminating at residual tolerance or after 25 iterations; if the budget is exhausted, IBR proceeds with the current iterate. Using batched NumPy on CPU, the full five-round update takes 18.24 ± 3.96 ms, within the 30 Hz control period. This demonstrates real-time KC-QP solves under changing inter-agent constraints without accelerator libraries in compute constrained setups.

B. 2D Quadcopter

We next consider a planar quadcopter with state $x = [p_x, p_z, \theta, v_x, v_z, \omega]$ and inputs given by collective thrust and torque in a $5\,\mathrm{m} \times 5\,\mathrm{m}$ workspace. We learn a 128-dimensional Koopman embedding using a three-hidden-layer, width-64 GELU encoder. The model is trained for 250 epochs using 40-step rollouts and a batch size of 8192.

TABLE I

Median solve times (ms)

Method	$H = 35$	$H = 45$	$H = 55$
Control constraints			
μMPC (CPU)	$\mathbf{0.023 \pm 0.027}$	$\mathbf{0.033 \pm 0.032}$	$\mathbf{0.040 \pm 0.065}$
μMPC (GPU)	0.129 ± 0.051	2.076 ± 0.845	2.082 ± 0.874
Control + obstacle constraints			
μMPC (CPU)	$\mathbf{0.277 \pm 0.115}$	$\mathbf{0.295 \pm 0.075}$	$\mathbf{0.322 \pm 0.099}$
μMPC (GPU)	0.996 ± 0.487	0.988 ± 0.265	0.949 ± 0.150

Table I shows that μMPC achieves 23–40 μs median CPU solve times with control constraints across the tested horizons. For these small condensed KC-QPs, GPU execution is slower because launch and synchronization overhead dominate the solve. With a centered 1 m-radius obstacle and start–goal pairs on opposing sides, we use an empirical model-error margin. Each of two outer updates relinearizes the nonlinear constraints about the previous control-induced trajectory; the inner primal–dual solve terminates when both residuals fall below 10^{-2} or after 150 iterations. Median CPU solve time remains below 0.33 ms across all horizons while meeting the prescribed residual tolerance.

C. 3D Quadcopter

We consider a 12-state quadcopter with four control inputs in a $5\,\mathrm{m} \times 5\,\mathrm{m} \times 5\,\mathrm{m}$ workspace. We use a 128-dimensional Koopman embedding with a three-hidden-layer, width-64 GELU encoder, trained for 250 epochs with batch size 4096 and 75-step rollouts.

We compare against acados/HPIPM [20], Koopman-BoxQP [18], OSQP [19] using both learned Condensed Koopman and analytical dynamics, and GPU-ADMM SQP [4]. Unlike trajectory-space SQP methods that update local dynamics online, μMPC reuses a fixed precomputed primal operator across MPC steps and constraint relinearizations, avoiding online refactorization. Table II shows 58–128 μs median CPU latency with control constraints. At $H = 100$,

TABLE II

MEDIAN SOLVE TIMES (MS)

Method	$H = 50$	$H = 75$	$H = 100$
Control constraints			
μMPC (CPU)	$\mathbf{0.058 \pm 0.070}$	$\mathbf{0.102 \pm 0.092}$	$\mathbf{0.128 \pm 0.102}$
μMPC (GPU)	0.213 ± 0.496	0.210 ± 0.161	0.189 ± 0.149
acados/HPIPM	0.183 ± 0.070	0.237 ± 0.087	0.289 ± 0.118
Koopman-BoxQP	3.218 ± 0.830	5.568 ± 1.416	8.229 ± 1.706
OSQP Condensed (CPU)	0.476 ± 0.078	0.975 ± 0.069	2.488 ± 0.243
OSQP Condensed (GPU)	4.312 ± 1.557	4.579 ± 2.745	6.685 ± 3.046
OSQP Analytical (CPU)	7.348 ± 4.676	15.408 ± 5.025	31.345 ± 9.291
OSQP Analytical (GPU)	14.510 ± 5.488	17.634 ± 6.762	24.532 ± 9.182
GPU-ADMM SQP (CPU)	7.239 ± 0.365	9.577 ± 0.392	13.877 ± 0.679
GPU-ADMM SQP (GPU)	6.201 ± 0.174	6.351 ± 0.182	6.276 ± 0.103
Control + obstacle constraints			
μMPC (CPU)	$\mathbf{0.454 \pm 0.597}$	$\mathbf{0.897 \pm 1.047}$	$\mathbf{1.339 \pm 1.869}$
μMPC (GPU)	1.771 ± 2.233	1.654 ± 2.258	1.365 ± 2.462
GPU-ADMM SQP (CPU)	6.569 ± 0.335	9.508 ± 0.181	14.051 ± 0.548
GPU-ADMM SQP (GPU)	6.174 ± 0.195	6.345 ± 0.158	6.207 ± 0.071

μMPC is 2.3× faster than acados/HPIPM, 19× faster than OSQP on the identical KC-QP, 64× faster than Koopman-BoxQP, and over 100× faster than GPU-ADMM SQP on CPU. The OSQP with our condensed dynamics comparison isolates the solver contribution that exploiting the fixed primal operator substantially reduces latency beyond condensation alone.

With a 1.25 m spherical obstacle, an empirical model-error margin, two constraint-linearization updates, and up to 150 primal-dual iterations, μMPC remains below 1.4 ms median CPU latency through $H = 100$, with the KC-QP feasibility residual remaining below the prescribed tolerance.

1) Edge Deployment: We next deploy the same obstacle-aware controller on a Raspberry Pi 5 and an NVIDIA Jetson Orin Nano to evaluate embedded execution. Table III shows

TABLE III

MEDIAN SOLVE TIMES (MS) ON EDGE DEVICES

Device	$H = 50$	$H = 75$	$H = 100$
Control constraints			
Raspberry Pi 5	0.576 ± 0.203	1.506 ± 0.258	3.102 ± 0.529
Jetson (CPU)	$\mathbf{0.448 \pm 0.297}$	$\mathbf{0.820 \pm 0.431}$	$\mathbf{1.314 \pm 0.756}$
Jetson (GPU)	1.760 ± 1.032	1.823 ± 0.800	1.860 ± 0.770
Control + obstacle constraints			
Raspberry Pi 5	7.255 ± 2.021	25.488 ± 3.049	42.590 ± 4.679
Jetson (CPU)	$\mathbf{2.369 \pm 2.430}$	$\mathbf{4.700 \pm 4.418}$	7.109 ± 8.987
Jetson (GPU)	5.213 ± 4.768	4.967 ± 4.033	$\mathbf{4.470 \pm 4.530}$

that control-constrained MPC remains below 3.2 ms on every edge configuration through $H = 100$. With obstacle constraints, the Jetson CPU remains below 7.2 ms, while the GPU becomes advantageous at longer horizons as the larger constrained problem better amortizes device overhead.

2) Differentiable-QP Ablation: We isolate the contributions of DAgger and differentiable-QP supervision. $\kappa_2(\bar{Q}_{\text{track}}^{1/2} G_x)$ is the tracking cost weighted spectral condition number and $\sigma_{\min}(\bar{Q}_{\text{track}}^{1/2} G_x)$ its smallest singular value; lower κ_2 and higher $\sigma_{\min}$ indicate a better-conditioned finite-horizon control map. DiffQP raises success from 3.7% to 97.7% at

TABLE IV

DIFFERENTIABLE QP AND DAGGER (†) ABLATION. HORIZON ENTRIES REPORT GOAL-REACHING SUCCESS (%) / MPC STEPS TO GOAL.

Method	$H = 75$	$H = 100$	$H = 125$
Base	3.71 / 155.5	6.84 / 160.5	6.84 / 215.0
Base +†	90.82 / 343.25	80.27 / 290.75	47.07 / 277.25
Base + diffQP	**97.66** / 328.5	92.19 / 516.6	85.55 / 508.7
Base +† + diffQP	97.46 **/ 299.0**	**96.88 / 266.5**	**95.70 / 285.5**
	$\kappa(\bar{Q}^{1/2} G_x) \downarrow$	$\sigma_{\min}(\bar{Q}^{1/2} G_x) \uparrow$	Control RMSE ↓
Base	6666	$2.73e{-4}$	0.814
Base +†	8612	$2.16e{-4}$	0.813
Base + diffQP	3551	$4.82e{-4}$	**0.747**
Base +† + diffQP	**1779**	$\mathbf{1.03 \times 10^{-3}}$	0.787

$H = 75$ and retains 85.6% at $H = 125$, while improving conditioning and control-recovery RMSE. Adding DAgger maintains over 95% success through $H = 125$ and reaches the goal in fewer MPC steps at long horizons. Although the combined model has slightly higher control RMSE than DiffQP alone, it has substantially better closed-loop performance, showing that DAgger complements DiffQP by adapting the model to controller-induced states. At batch size 4096, DiffQP incurs modest training overhead, increasing per-epoch time from 21.97 to 26.82 s and peak VRAM from 2082 to 2224 MiB.

3) Comparison with TinyMPC: For the unconstrained KC-QP, we additionally apply the Woodbury identity when $\tilde{G} = \bar{Q}_y^{1/2} G_y$ has fewer rows than Hn_u, reducing the precomputed fixed system dimension. This gives an exact solve using only matrix–vector products online. The Woodbury form

TABLE V

MEAN / MEDIAN ± STD. RTI LATENCY (MS) VERSUS TINYMPC

Method	$H = 50$	$H = 60$	$H = 70$
μMPC (Woodbury)	$\mathbf{0.029/0.023 \pm 0.013}$	$\mathbf{0.035/0.029 \pm 0.017}$	$\mathbf{0.045/0.037 \pm 0.019}$
TinyMPC	$0.853/0.036 \pm 2.280$	$0.722/0.038 \pm 2.188$	$0.785/0.042 \pm 2.380$

requires 23–37 μs median latency, comparable to TinyMPC's median runtime but with substantially lower mean latency and variance. The Woodbury result is an unconstrained exact KC-QP solve, whereas TinyMPC retains actuator bounds and uses analytical hover-linearized dynamics with a different terminal objective; the comparison therefore characterizes closed-loop computational cost rather than strictly identical-QP solver performance.

4) Crazyflie Deployment: We deploy μMPC on a Crazyflie Cortex–M4F for the point-to-point task in Fig. 1B using a 128-dimensional Koopman model with $H = 75$ ($q = 300$, 1.875 s horizon). This deployment uses an unconstrained KC-QP, so the optimal action sequence is obtained directly from the fixed quadratic system without primal–dual iterations. For memory efficiency, the MCU stores precompiled `float32` quantities and a packed Cholesky factor. The controller requires 601.7 kB of immutable storage and 8.7 kB of runtime workspace; total firmware size is 948.4 kB.

After hover-thrust calibration, MPC commands normalized thrust and body rates. The MCU averages 20.08 ms over 100 benchmark cases with no 25 ms deadline violations and

24.57 ms during flight. The full radio-mediated experimental loop runs at approximately 15 Hz due to communication, state acquisition, Vicon forwarding, and telemetry.

5) Scalable Multi-Agent Quadcopter Planning: We evaluate Jacobi IBR in a 25m cube, reusing the original Koopman model without retraining by planning in moving 5m local frames centered on each robot. Each quadrotor treats other agents' predicted trajectories as time-varying collision constraints and all KC-QPs are solved in one GPU batch. We use up to 2 IBR sweeps, 1 SQP iteration, and 150 PD iterations per KC-QP, terminating early upon convergence and accepting the current iterate when the budget is exhausted. Goal % denotes agents reaching their assigned goal; Collision % denotes agents experiencing at least one collision.

TABLE VI

GPU-BATCHED, MULTI-QUADROTOR MPC AT SCALE

Metric	50 Quadrotors	100 Quadrotors	150 Quadrotors	200 Quadrotors
Solve (ms)	12.089 ± 0.017	17.347 ± 0.156	23.020 ± 0.754	32.117 ± 0.913
Goal %	$100.0 \pm 0.0\%$	$99.760 \pm 0.427\%$	$99.173 \pm 0.633\%$	$98.420 \pm 1.137\%$
Collision %	$0.0 \pm 0.0\%$	$1.240 \pm 1.305\%$	$1.867 \pm 1.373\%$	$2.580 \pm 1.278\%$

Scaling from 50 to 200 quadrotors increases batched MPC/IBR latency from 12.1 to 32.1 ms (2.66× for 4× more agents), while maintaining over 30 Hz operation and above 98% goal reaching. Collision incidence rises to 2.58% at $D = 200$. At large scales, the fixed iteration budget trades full best-response convergence for real-time updates, so residual collision-constraint violations may remain. KC-QP is well suited to IBR because inter-agent constraints change each round while the primal operator remains fixed.

D. Quadruped

1) Control Constrained Planning: We evaluate whole-body quadruped control using a 4096-dimensional Koopman model of the 61-dimensional robot state, comprising floating-base pose and velocity, joint states, foot positions, and ground-reaction forces, with 12 joint-torque inputs. The three-hidden-layer, width-512 GELU encoder is trained for 100 epochs with batch size 2500 and 55-step rollouts. To avoid out-of-distribution analytical gait references, we distill a SAC locomotion policy into a learned reference generator supplying joint states, foot heights, and vertical ground-reaction forces for KC-QP tracking. Fixed base height and commanded linear and angular velocities are also tracked. We compare against MPX's primal–dual iLQR [3] and GPU-ADMM SQP [4] using identical reference generation and torque limits. Reported latencies include reference generation for all methods and state encoding for μMPC.

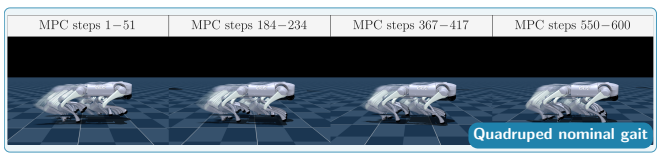

Fig. 3. Nominal Go2 locomotion under μMPC over a closed-loop rollout.

Table VII shows that at $H = 100$, μMPC is 1.5× faster than MPX and 4.7× faster than GPU-ADMM SQP; the shared reference generator contributes 2.71/5.72/8.53 ms at $H = 25/65/100$. On the Nvidia Jetson Orin Nano, μMPC runs in 24–38 ms, versus 28–84 ms for MPX and 173–637 ms for GPU-ADMM SQP. All control-constrained solves terminate within the 150-iteration primal–dual budget at the standard 10^{-2} residual tolerance.

TABLE VII

MEDIAN SOLVE TIMES (MS)

Method	$H = 25$	$H = 65$	$H = 100$
Control constraints			
μMPC	$\mathbf{4.472 \pm 0.253}$	$\mathbf{8.760 \pm 0.356}$	$\mathbf{13.295 \pm 0.456}$
MPX	6.793 ± 0.331	13.464 ± 0.483	20.062 ± 0.555
GPU-ADMM SQP	23.743 ± 1.459	47.111 ± 2.560	62.951 ± 3.821
Deployment on Jetson with Control Constraints			
μMPC	$\mathbf{24.467 \pm 1.887}$	$\mathbf{32.185 \pm 3.231}$	$\mathbf{38.110 \pm 3.955}$
MPX	28.437 ± 1.550	70.881 ± 3.269	84.197 ± 2.992
GPU-ADMM SQP	173.023 ± 6.405	409.777 ± 6.475	637.463 ± 8.696

2) Whole-Body Obstacle-Avoidance Filtering: We filter the same SAC policy, operating within a learned simulator, for state/torque reference and KC-QP obstacle constraints. This harder constrained problem uses up to 750 primal–dual iterations and a relaxed 5×10^{-2} residual tolerance to preserve the real-time rate. Across our evaluated rollouts, the filtered controller avoids the obstacle crossed by the nominal policy (Fig. 1A). At $H = 100$, end-to-end latency is 28.18 ms: 7.74 ms reference generation, 0.22 ms encoding, and 20.13 ms KC-QP optimization. Constraint tightenings are empirical rather than uncertainty-calibrated; formal robust margins remain future work.

E. Linear World Models for Visual Planning

We evaluate all planners on pixel-based bimanual rope manipulation and Reacher using random workspace start–goal pairs, rather than LeWM [9] and Fast-LeWM's [10] 25-step same-trajectory goal sampling. Images are encoded by a custom ViT into a 12-dimensional embedding, augmented with first- and second-order differences to form $s_t = [e_t, \Delta e_t, \Delta^2 e_t] \in \mathbb{R}^{36}$, and lifted to a 2048-dimensional Koopman model with linear dynamics. For rope manipulation, $u_t = [w_t, h_t, r_t]$ specifies gripper separation, height, and fore–aft reach and is executed through inverse kinematics. We compile the KC-QP offline, so the 2048-dimensional latent trajectory never appears in the online decision variables; planning optimizes only the action sequence with explicit action bounds enforced as linear inequalities.

TABLE VIII

END-TO-END PLANNING PERFORMANCE AND SOLVE TIMES (MS)

Metric	**μMPC**	LeWM	Fast-LeWM
Bi-Manual Rope			
Solve time (ms)	$\mathbf{0.725 \pm 0.164}$	820.541 ± 42.092	559.063 ± 41.601
Success rate	**94%**	24%	46%
Reacher			
Solve time (ms)	$\mathbf{0.427 \pm 0.098}$	814.2 ± 30.9	558.8 ± 29.2
Success rate	**62%**	4%	8%

Table VIII shows that μMPC plans in 0.725 ms on bimanual rope manipulation and 0.427 ms on Reacher, while

achieving 94% and 62% success, compared with 24%/4% for LeWM and 46%/8% for Fast-LeWM. All evaluated KC-QPs reach the prescribed primal–dual residual tolerances within the real-time iteration budget and respect the imposed action bounds. On the Jetson GPU, the same planners run in approximately 33.060 ± 1.107 ms on both tasks, sustaining about 30 Hz even on edge hardware. These results demonstrate that scaling learned visual representations to 2048 dimensions preserves fast online optimization without requiring repeated latent rollouts or sampling-based search.

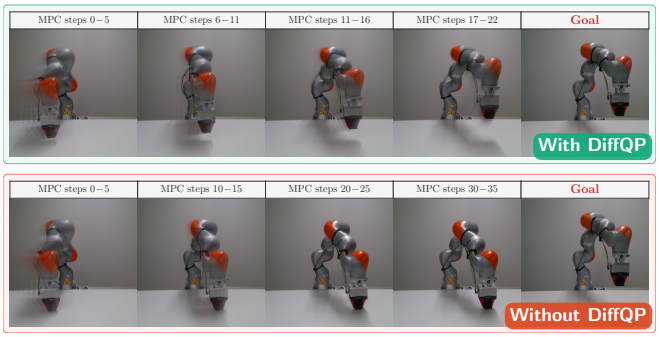

Fig. 4. Real-world cube manipulation with and without diffQP supervision.

1) Hardware Cube Manipulation: We validate the same formulation on real cube manipulation, where μMPC commands Cartesian end-effector targets $u_t = [x_t, y_t, z_t]$ through inverse kinematics to match a goal image taking 0.483 ± 0.130 ms per solve. As shown in Fig. 4, differentiable-QP supervision recovers the vertical control authority required to reach the elevated target, whereas prediction-only training remains below the goal height despite approaching the correct planar configuration.

VI. CONCLUSION

We presented μMPC, a framework that combines scalable Koopman dynamics learning with fast constrained MPC. By compiling finite-horizon predictions into action-only KC-QPs with a reusable primal operator, μMPC reduces online primal-dual optimization to matrix-vector products and projections, invariant under changing goals and constraints. Lightweight differentiable control-recovery QPs enable fast, batched supervision and scalable DAgger-style data aggregation, directly training dynamics for finite-horizon control. Across experiments, μMPC achieves millisecond-scale whole-body control with 4096-dimensional Koopman models, submillisecond planning with 2048-dimensional linear world models, long horizon MPC onboard Crazyflie MCU, and real-time planning for 200 quadrotors.

Future work should focus on principled model selection and improved constraint handling. Our reported encoder capacities, lifted dimensions, and training horizons were selected through empirical tuning and retraining, motivating scaling laws relating these choices to prediction and closed-loop performance. In very demanding cases, fixed-budget solves may retain residual constraint violations, incorporating constraints into training could improve constraint response with fewer primal-dual iterations. Current model-error margins also remain empirical rather than formally calibrated.

REFERENCES

[1] K. Nguyen, S. Schoedel, A. Alavilli, B. Plancher, and Z. Manchester, "TinyMPC: Model-predictive control on resource-constrained microcontrollers," in *IEEE ICRA*, 2024.

[2] A. Du, E. Adabag, G. Bravo-Palacios, and B. Plancher, "GATO: Gpu-accelerated and batched trajectory optimization for scalable edge model predictive control," in *IEEE ICRA*, 2026.

[3] L. Amatucci, J. Sousa-Pinto, G. Turrisi, D. Orban, V. Barasuol, and C. Semini, "Primal-dual ilqr for gpu-accelerated learning and control in legged robots," *IEEE RAL*, vol. 11, no. 1, pp. 1010–1017, 2026.

[4] J. Fang and G. Chou, "Safe large-scale robust nonlinear mpc in milliseconds via reachability-constrained system level synthesis on the gpu," in *RSS*, 2026.

[5] G. Zhou, H. Pan, Y. LeCun, and L. Pinto, "DINO-WM: World models on pre-trained visual features enable zero-shot planning," in *ICML*, ser. PMLR, vol. 267. PMLR, 2025, pp. 79 115–79 135.

[6] D. Nath, A. Srinivasan, H. Yin, R. Jiang, J. Fang, and G. Chou, "Pixels to proofs: Probabilistically-safe latent world model control via parallel conformal robust mpc," *arXiv preprint arXiv:2606.15594*, 2026.

[7] S. Zhan, C.-Y. Chiu, A. P. Leeman, and G. Chou, "Robustly constrained dynamic games for uncertain nonlinear dynamics," in *ICRA*. IEEE, 2026.

[8] G. Foderaro, S. Ferrari, and T. A. Wettergren, "Distributed optimal control for multi-agent trajectory optimization," *Automatica*, vol. 50, no. 1, pp. 149–154, 2014.

[9] L. Maes, Q. L. Lidec, D. Scieur, Y. LeCun, and R. Balestriero, "Leworldmodel: Stable end-to-end joint-embedding predictive architecture from pixels," 2026.

[10] Y. Gao and X. Xu, "Fast leworldmodel," 2026.

[11] M. Korda and I. Mezić, "Linear predictors for nonlinear dynamical systems: Koopman operator meets model predictive control," *Automatica*, vol. 93, pp. 149–160, 2018.

[12] I. Mezić, J. Cortés, K. Worthmann, M. Lazar, and A. Lederer, "Koopman operator theory: fundamentals, control, and applications," *arXiv preprint arXiv:2607.01819*, 2026.

[13] B. Amos and J. Z. Kolter, "OptNet: Differentiable optimization as a layer in neural networks," in *ICML*, ser. PMLR, vol. 70. PMLR, 2017, pp. 136–145.

[14] H. Shi and M. Q.-H. Meng, "Deep koopman operator with control for nonlinear systems," *IEEE RAL*, vol. 7, no. 3, pp. 7700–7707, 2022.

[15] F. Li, A. Abuduweili, Y. Sun, R. Chen, W. Zhao, and C. Liu, "Continual learning and lifting of koopman dynamics for linear control of legged robots," in *L4DC*, ser. PMLR, vol. 283. PMLR, 2025, pp. 136–148.

[16] S. Jung, A. Abuduweili, J. Li, and C. Liu, "Whole-body safe control of robotic systems with koopman neural dynamics," 2026.

[17] L. Wu, K. Ganko, and R. D. Braatz, "Time-certified input-constrained nmpc via koopman operator," in *IFAC-PapersOnLine*, vol. 58, no. 18, 2024, pp. 335–340.

[18] L. Wu, W. G. Y. Tan, R. Braatz, and J. Drgona, "Koopman-boxqp: Solving large-scale nmpc at khz rates," in *L4DC*, ser. PMLR, vol. 331. PMLR, 2026, pp. 1791–1803.

[19] B. Stellato, G. Banjac, P. Goulart, A. Bemporad, and S. Boyd, "OSQP: An operator splitting solver for quadratic programs," *Mathematical Programming Computation*, vol. 12, no. 4, pp. 637–672, 2020.

[20] R. Verschueren, G. Frison, D. Kouzoupis, J. Frey, N. van Duijkeren, A. Zanelli, B. Novoselnik, T. Albin, R. Quirynen, and M. Diehl, "acados—a modular open-source framework for fast embedded optimal control," *Mathematical Programming Computation*, vol. 14, no. 1, pp. 147–183, 2022.

[21] A. Agrawal, B. Amos, S. Barratt, S. Boyd, S. Diamond, and J. Z. Kolter, "Differentiable convex optimization layers," in *NeurIPS*, vol. 32, 2019, pp. 9558–9570.

[22] B. Amos, I. D. J. Rodriguez, J. Sacks, B. Boots, and J. Z. Kolter, "Differentiable mpc for end-to-end planning and control," 2018.

[23] R. Tao, S. Cheng, X. Wang, S. Wang, and N. Hovakimyan, "DiffTune-MPC: Closed-loop learning for model predictive control," *IEEE RAL*, vol. 9, no. 8, pp. 7294–7301, 2024.

[24] M. Retchin, B. Amos, S. L. Brunton, and S. Song, "Koopman constrained policy optimization: A koopman operator theoretic method for differentiable optimal control in robotics," in *ICML Workshop on Differentiable Almost Everything*, 2023.

[25] D. Hafner, T. Lillicrap, I. Fischer, R. Villegas, D. Ha, H. Lee, and J. Davidson, "Learning latent dynamics for planning from pixels," in *ICML*, ser. PMLR, vol. 97. PMLR, 2019, pp. 2555–2565.

[26] N. Hansen, H. Su, and X. Wang, "TD-MPC2: Scalable, robust world models for continuous control," in *ICLR*, 2024.
[27] D. Hafner, J. Pasukonis, J. Ba, and T. Lillicrap, "Mastering diverse control tasks through world models," *Nature*, vol. 640, no. 8059, pp. 647–653, 2025.
[28] A. Sommer and J. Schilling, "Reinforced planning with latent world models," 2026.